ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

ассистент		Д.А. Кочин
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №2

РАЗРАБОТКА МНОГОПОТОЧНОГО ПРИЛОЖЕНИЯ СРЕДСТВАМИ POSIX В ОС LINUX ИЛИ MAC OS

по курсу: ОПЕРАЦИОННЫЕ СИСТЕМЫ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. №	4236		А.В. Панченко
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2024

Цель работы:

Знакомство с многопоточным программированием и методами синхронизации потоков средствами POSIX.

Индивидуальное задание:

1. С помощью таблицы вариантов заданий выбрать граф запуска потоков в соответствии с номером варианта. Вершины графа являются точками запуска/завершения потоков, дугами обозначены сами потоки. Длину дуги следует интерпретировать как ориентировочное время выполнения потока. В процессе своей работы каждый поток должен в цикле выполнять два действия:
   i. выводить букву имени потока в консоль;
   ii. вызывать функцию computation() для выполнения вычислений, требующих задействования ЦП на длительное время. Эта функция уже написана и подключается из заголовочного файла lab2.h, изменять ее не следует.
2. В соответствии с вариантом выделить на графе две группы с выполняющимися параллельно потоками. В первой группе потоки не синхронизированы, параллельное выполнение входящих в группу потоков происходит за счет планировщика задач (см. примеры 1 и 2). Вторая группа синхронизирована семафорами и потоки внутри группы выполняются в строго зафиксированном порядке: входящий в групу поток передает управление другому потоку после каждой итерации цикла (см. пример 3 и задачу производителя и потребителя). Таким образом потоки во второй группе выполняются в строгой очередности.
3. С использованием средств POSIX реализовать программу для последовательно-параллельного выполнения потоков в ОС Linux или Mac OS X. Запрещается использовать какие-либо библиотеки и модули, решающие задачу кроссплатформенной разработки многопоточных приложений (std::thread, Qt Thread, Boost Thread и т.п.), а также функции

приостановки выполнения программы за исключением pthread_yield(). Для этого необходимо написать код в файле lab2.cpp:

i. Функция unsigned int lab2_thread_graph_id() должна возвращать номер графа запуска потоков, полученный из таблицы вариантов заданий.
ii. Функция const char* lab2_unsynchronized_threads() должна возвращать строку, состоящую из букв потоков, выполняющихся параллельно без синхронизации (см. примеры в файлах lab2.cpp и lab2_ex.cpp).
iii. Функция const char* lab2_sequential_threads() должна возвращать строку, состоящую из букв потоков, выполняющихся параллельно в строгой очередности друг за другом (см. примеры в файлах lab2.cpp и lab2_ex.cpp).
iv. Функция int lab2_init() заменяет собой функцию main(). В ней необходимо реализовать запуск потоков, инициализацию вспомогательных переменных (мьютексов, семафоров и т.п.). Перед выходом из функции lab2_init() необходимо убедиться, что все запущенные потоки завершились. Возвращаемое значение: 0 - работа функции завершилась успешно, любое другое числовое значение - при выполнении функции произошла критическая ошибка.
v. Добавить любые другие необходимые для работы программы функции, переменные и подключаемые файлы.
vi. Создавать функцию main() не нужно. В проекте уже имеется готовая функция main(), изменять ее нельзя. Она выполняет единственное действие: вызывает функцию lab2_init().
vii. Не следует изменять какие-либо файлы, кроме lab2.cpp. Также не следует создавать новые файлы и писать в них код, поскольку код из этих файлов не будет использоваться во время тестирования.

4. Подготовить отчет о выполнении лабораторной работы и загрузить его под именем report.pdf в репозиторий. В случае использования системы компьютерной верстки LaTeX также загрузить исходный файл report.tex.

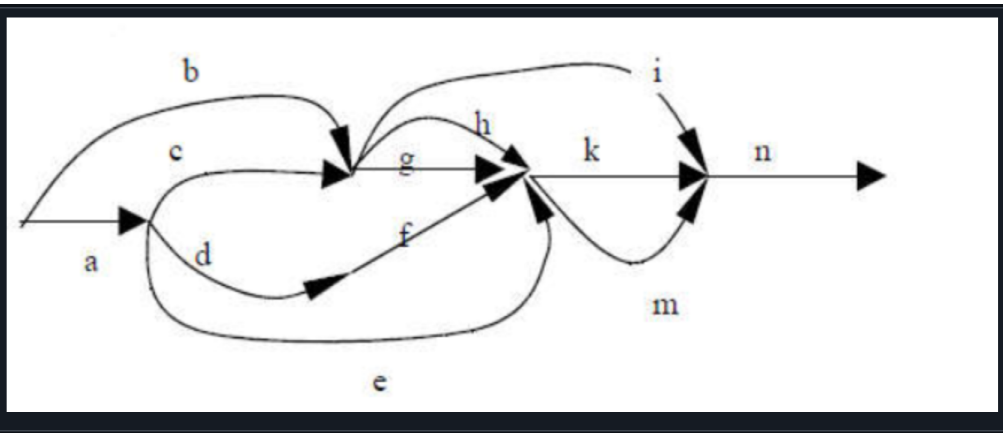
Последовательное выполнение потоков может обеспечиваться как за счет использования семафоров, так и с помощью функции pthread_join(). Запускать потоки можно все сразу в функции lab2_init(), а можно и по одному (или группами) из других потоков. Количество запускаемых потоков должно быть равно количеству дуг на графе плюс один (для потока main). Запрещается завершать поток в конце интервала, а затем заново его запускать.

В процессе своей работы каждый поток выводит свою букву в консоль. Оценка правильности выполнения лабораторной работы осуществляется следующим образом. Если потоки **a** и **b** согласно графу должны выполняться одновременно (параллельно), то в консоли должна присутствовать последовательность вида **abababab** (или схожая, например, **aabbba**); если потоки выполняются последовательно, то в консоли присутствует последовательность вида **aaaaabbbbbb**, причем после появления первой буквы **b**, буква **a** больше не должна появиться в консоли.

Количество букв, выводимых каждым потоком в консоль, должно быть пропорционально числу интервалов (длине дуги), соответствующей данному потоку на графе. При этом количество символов, выводимых в консоль каждым из потоков, должно быть не меньше чем 3Q и не больше чем 5Q, где Q - количество интервалов на графе, в течении которых выполняется поток. Множитель перед величиной Q следует выбрать одинаковым для всех потоков, задав его равным 3, 4 или 5. Ожидается, что на каждом интервале своей работы поток выводит одинаковое количество символов в консоль.

Номер варианта	Номер графа запуска потоков	Интервалы с несинхронизированными потоками	Интервалы с чередованием потоков
18	2	bcde	ikm

Граф запуска потоков:

2	

Результат выполнения работ:

```
parallels@ubuntu-linux-22-04-02-desktop: ~/lab3
parallels@ubuntu-linux-22-04-02-desktop:~$ cd lab3
parallels@ubuntu-linux-22-04-02-desktop:~/lab3$ cd os-task2-Oblachko-Js/
parallels@ubuntu-linux-22-04-02-desktop:~/lab3/os-task2-Oblachko-Js$ cd test
parallels@ubuntu-linux-22-04-02-desktop:~/lab3/os-task2-Oblachko-Js/test$ ./runTests
[==========] Running 5 tests from 1 test case.
[----------] Global test environment set-up.
[----------] 5 tests from lab2_tests
[ RUN      ] lab2_tests.tasknumber
TASKID is 2
[       OK ] lab2_tests.tasknumber (0 ms)
[ RUN      ] lab2_tests.unsynchronizedthreads
Unsynchronized threads are bcde
[       OK ] lab2_tests.unsynchronizedthreads (0 ms)
[ RUN      ] lab2_tests.sequentialthreads
Sequential threads are ikm
[       OK ] lab2_tests.sequentialthreads (0 ms)
[ RUN      ] lab2_tests.threadsync
Output for graph 2 is: baabbaebdcbbecdedceifhgegifhighefikmikmikmnnn
Intervals are:
baabba
ebdcbbecdedc
eifhgegifhighef
ikmikmikm
nnn
[       OK ] lab2_tests.threadsync (1109 ms)
[ RUN      ] lab2_tests.concurrency
Completed 0 out of 100 runs.
Completed 20 out of 100 runs.
Completed 40 out of 100 runs.
Completed 60 out of 100 runs.
Completed 80 out of 100 runs.
[       OK ] lab2_tests.concurrency (114194 ms)
[----------] 5 tests from lab2_tests (115303 ms total)

[----------] Global test environment tear-down
[==========] 5 tests from 1 test case ran. (115303 ms total)
[  PASSED  ] 5 tests.
parallels@ubuntu-linux-22-04-02-desktop:~/lab3/os-task2-Oblachko-Js/test$
```

Исходный код выполнения программы с коментариями:

```
#include "lab2.h"
#include <cstring>
#include <semaphore.h>

// Возвращает идентификатор графа задачи
unsigned int lab2_thread_graph_id()
{
   return 2;
}

// Указывает потоки, которые выполняются без синхронизации
const char* lab2_unsynchronized_threads()
{
   return "bcde";
}

// Указывает потоки, которые выполняются последовательно
const char* lab2_sequential_threads()
{
   return "ikm";
}

using namespace std;

pthread_mutex_t mutexUnsynchronizedEdge; // Мьютекс для управления неблокирующими задачами

// Функция для выполнения задачи без синхронизации
void handleTaskUnsynchronized(const string &edgeId)
```

```
{
    for (int i = 0; i < 3; ++i) {
        pthread_mutex_lock(&mutexUnsynchronizedEdge);    //    Блокировка
мьютекса
        std::cout << edgeId << std::flush; // Вывод идентификатора задачи
        pthread_mutex_unlock(&mutexUnsynchronizedEdge);                    //
Разблокировка мьютекса
        computation(); // Выполнение вычислений
    }
}

// Функция для выполнения задачи с синхронизацией
void handleTaskSynchronized(const string &edgeId,
                            sem_t &currentTask,
                            sem_t &nextTask)
{
    for (int i = 0; i < 3; ++i) {
        sem_wait(&currentTask); // Ожидание семафора текущей задачи
        std::cout << edgeId << std::flush; // Вывод идентификатора задачи
        sem_post(&nextTask); // Разрешение выполнения следующей задачи
        computation(); // Выполнение вычислений
    }
}

// Объявление семафоров для синхронизации задач
sem_t mutexEdgeIdISync, mutexEdgeIdKSync, mutexEdgeIdMSync,
    mutexEdgeIdFSync, mutexEdgeIdESync, mutexEdgeIdB, mutexEdgeIdE,
mutexEdgeIdI;

void *edgeIdE(void *ptr);
```

```
void *edgeIdC(void *ptr);
void *edgeIdA(void *ptr);
void *edgeIdI(void *ptr);
void *edgeIdD(void *ptr);
void *edgeIdG(void *ptr);
void *edgeIdF(void *ptr);
void *edgeIdH(void *ptr);

// Функция выполнения задачи потока B
void *edgeIdB(void *ptr) {
    pthread_t idA;
    pthread_create(&idA, 0, edgeIdA, NULL); // Создание потока A
    handleTaskUnsynchronized("b"); // Выполнение задачи B
    sem_wait(&mutexEdgeIdB); // Ожидание завершения B
    pthread_t idE;
    pthread_create(&idE, 0, edgeIdE, NULL); // Создание потока E
    pthread_t idD;
    pthread_create(&idD, 0, edgeIdD, NULL); // Создание потока D
    pthread_t idC;
    pthread_create(&idC, 0, edgeIdC, NULL); // Создание потока C
    handleTaskUnsynchronized("b"); // Выполнение задачи B
    sem_wait(&mutexEdgeIdB); // Ожидание завершения B
    sem_wait(&mutexEdgeIdB); // Ожидание завершения B
    sem_wait(&mutexEdgeIdB); // Ожидание завершения B
    sem_post(&mutexEdgeIdE); // Разрешение выполнения задачи E

    pthread_t idI;
    pthread_create(&idI, 0, edgeIdI, NULL); // Создание потока I
    pthread_t idF;
    pthread_create(&idF, 0, edgeIdF, NULL); // Создание потока F
```

```
    pthread_t idG;
    pthread_create(&idG, 0, edgeIdG, NULL); // Создание потока G
    pthread_t idH;
    pthread_create(&idH, 0, edgeIdH, NULL); // Создание потока H
    pthread_join(idI, NULL); // Ожидание завершения потока I
    return ptr;
}

// Функция выполнения задачи потока A
void *edgeIdA(void *ptr) {
    handleTaskUnsynchronized("a"); // Выполнение задачи A
    sem_post(&mutexEdgeIdB); // Разрешение выполнения задачи B
    return ptr;
}

// Функции выполнения задач для других потоков C, D, E, F, G, H аналогичны

// Функция выполнения задачи потока I
void *edgeIdI(void *ptr)
{
    handleTaskUnsynchronized("i"); // Выполнение задачи I

    // Ожидание завершения синхронизированных задач
    sem_wait(&mutexEdgeIdI);
    sem_wait(&mutexEdgeIdI);
    sem_wait(&mutexEdgeIdI);
    sem_wait(&mutexEdgeIdI);

    // Создание и выполнение задач M и K
```

```
    pthread_t idM;
    pthread_create(&idM, NULL, edgeIdM, NULL);
    pthread_t idK;
    pthread_create(&idK, NULL, edgeIdK, NULL);

    // Синхронизированное выполнение задачи I
    handleTaskSynchronized("i", mutexEdgeIdISync, mutexEdgeIdKSync);

    sem_wait(&mutexEdgeIdI); // Ожидание завершения задачи I

    // Создание и выполнение задачи N
    pthread_t idN;
    pthread_create(&idN, 0, edgeIdN, NULL);
    pthread_join(idN, NULL); // Ожидание завершения потока N
    return ptr;
}

// Функции для других потоков аналогичны

// Инициализация синхронизации и запуск потоков
int lab2_init()
{
    pthread_mutex_init(&mutexUnsynchronizedEdge,        NULL);        //
Инициализация мьютекса
    sem_init(&mutexEdgeIdB, 0, 0); // Инициализация семафоров
    sem_init(&mutexEdgeIdI, 0, 0);
    sem_init(&mutexEdgeIdE, 0, 0);
    sem_init(&mutexEdgeIdKSync, 0, 0);
    sem_init(&mutexEdgeIdFSync, 0, 0);
```

```
    sem_init(&mutexEdgeIdISync, 0, 1); // Начальное значение для синхронизации I
    sem_init(&mutexEdgeIdMSync, 0, 0);
    sem_init(&mutexEdgeIdESync, 0, 0);

    pthread_t idEdgeB;
    pthread_create(&idEdgeB, 0, edgeIdB, NULL); // Запуск потока B
    pthread_join(idEdgeB, NULL); // Ожидание завершения потока B

    // Очистка ресурсов синхронизации
    pthread_mutex_destroy(&mutexUnsynchronizedEdge);
    sem_destroy(&mutexEdgeIdI);
    sem_destroy(&mutexEdgeIdB);
    sem_destroy(&mutexEdgeIdE);
    sem_destroy(&mutexEdgeIdISync);
    sem_destroy(&mutexEdgeIdKSync);
    sem_destroy(&mutexEdgeIdESync);
    sem_destroy(&mutexEdgeIdFSync);
    sem_destroy(&mutexEdgeIdMSync);

    return 0;
}
```

Выводы:

Были изучены принципы многопоточного программирования с использованием мьютексов и семафоров для синхронизации потоков. Реализованы методы управления потоками для последовательного и параллельного выполнения задач, что позволило освоить основы координации потоков и предотвращения гонок данных в многозадачных средах.